

Name: J O Joanna Divine

Reg No: 192572003

Course Code: CSA0918

Systematic Investigation: Relating Operations to Collection Complexity

Question: A program works correctly with small data but becomes slow with one million elements. Prepare a systematic investigation that relates operations performed to the complexity of the selected collections.

Background

Correctness and performance are separate concerns: a program can produce the right answer on 50 test records and still be unusable on a million, because the collection chosen fixes the growth rate of every operation performed on it. A method call that looks identical in the source code (`list.contains(x)`) can be $O(1)$ or $O(n)$ purely depending on whether list is a `HashSet` or an `ArrayList`. The investigation below traces the reported symptom back to the specific operation and the specific collection responsible.

1. Step-by-Step Investigation Plan

- 1) Reproduce the issue with a controllable input size, and record actual wall-clock time for a few sizes (e.g. 1,000 / 10,000 / 100,000) instead of relying on the vague report of "slow at one million".
- 2) List every collection used in the slow code path (`ArrayList`, `LinkedList`, `HashMap`, `HashSet`, `TreeMap`, etc.) and every operation called on each (`add`, `get`, `contains`, `remove`, `put`).
- 3) Look up the documented time complexity of each operation for the collection actually used — not the collection interface in general (`List.contains()` is $O(n)$ for `ArrayList` but the same method name is $O(1)$ average for a `HashSet`-backed structure).
- 4) Identify operations called inside a loop. An $O(n)$ operation called once per outer-loop iteration turns an $O(n)$ loop into $O(n^2)$ overall — this is the pattern most responsible for "fine on small data, unusable on large data".
- 5) Form a hypothesis: name the exact line and the exact collection you believe is the bottleneck, based on step 4, before changing any code.
- 6) Benchmark the suspected operation in isolation across increasing N and compute the ratio in runtime each time N doubles. A ratio that stays near 2x indicates $O(n)$; a ratio that approaches 4x indicates $O(n^2)$; a ratio that stays flat indicates $O(1)$ or $O(\log n)$.
- 7) Compare the measured ratio against the theoretical complexity from step 3 to confirm (or reject) the hypothesis.
- 8) Replace the offending collection/operation with one whose complexity matches the access pattern actually needed (e.g. replace `ArrayList.contains()` with `HashSet.contains()` when only membership testing is required).
- 9) Re-run the same benchmark on the corrected version and confirm the growth rate has changed as predicted.
- 10) Document the finding: operation, collection, old complexity, new complexity, and measured improvement.

2. Complexity Reference (operations relevant to this investigation)

Collection	Operation	Time Complexity	Reason
ArrayList	get(index)	O(1)	direct index into backing array
ArrayList	add(element) at end	O(1) amortised	array append, occasional resize
ArrayList	contains(x) / indexOf(x)	O(n)	must scan elements one by one
ArrayList	remove(Object)	O(n)	scan to find, then shift remaining elements
LinkedList	get(index)	O(n)	must walk node-by-node from an end
LinkedList	addFirst / addLast	O(1)	direct pointer update, no shifting
HashSet / HashMap	add / put	O(1) average	hashed directly to a bucket
HashSet / HashMap	contains / get	O(1) average	hashed directly to a bucket
TreeSet / TreeMap	add / contains / get	O(log n)	balanced tree traversal

3. Investigation Code

The class below reproduces the pattern that most often causes this exact symptom: checking for duplicates with `list.contains()` inside a loop (ArrayList, Part A) versus `set.contains()` inside the same loop (HashSet, Part B). Both are then extrapolated / measured at $N = 1,000,000$ to confirm the theory.

```
import java.util.*;

public class DuplicateCheckBenchmark {

    // Slow version: ArrayList.contains() is a linear scan -> O(n) per
    // check -> O(n^2) overall
    static long timeArrayListDedup(int n) {
        List<Integer> list = new ArrayList<>();
        long start = System.nanoTime();
        for (int i = 0; i < n; i++) {
            if (!list.contains(i)) { // O(n) scan on every call
                list.add(i);
            }
        }
        long end = System.nanoTime();
        return (end - start) / 1_000_000; // ms
    }

    // Fast version: HashSet.contains() is O(1) average -> O(n) overall
    static long timeHashSetDedup(int n) {
        Set<Integer> set = new HashSet<>();
        long start = System.nanoTime();
```

```

        for (int i = 0; i < n; i++) {
            if (!set.contains(i)) {    // O(1) average lookup
                set.add(i);
            }
        }
        long end = System.nanoTime();
        return (end - start) / 1_000_000; // ms
    }

    public static void main(String[] args) {
        // JVM warm-up (JIT compilation) so the first real measurement is
        not skewed
        timeArrayListDedup(1000);
        timeHashSetDedup(1000);

        int[] sizesSmallToMedium = {4000, 8000, 16000, 32000};

        System.out.println("=====
        ==");
        System.out.println(" PART A: ArrayList.contains() based de-
        duplication");
        System.out.println(" (Operation used: contains() -> O(n) scan per
        call)");

        System.out.println("=====
        ==");
        System.out.printf("%-12s%-15s%-15s\n", "N", "Time (ms)", "Ratio vs
        prev");
        long prev = -1;
        for (int n : sizesSmallToMedium) {
            long t = timeArrayListDedup(n);
            String ratio = (prev <= 0) ? "-" : String.format("%.2fx",
            (double) t / prev);
            System.out.printf("%-12d%-15d%-15s\n", n, t, ratio);
            prev = t;
        }

        System.out.println();

        System.out.println("=====
        ==");
        System.out.println(" PART B: HashSet.contains() based de-
        duplication");
        System.out.println(" (Operation used: contains() -> O(1) average
        per call)");

        System.out.println("=====
        ==");
        int[] sizesFull = {4000, 8000, 16000, 32000, 1_000_000};
        System.out.printf("%-12s%-15s%-15s\n", "N", "Time (ms)", "Ratio vs
        prev");
        prev = -1;
        for (int n : sizesFull) {

```

```

        long t = timeHashSetDedup(n);
        String ratio = (prev <= 0) ? "-" : String.format("%.2fx",
(double) t / prev);
        System.out.printf("%-12d%-15d%-15s%n", n, t, ratio);
        prev = t;
    }

    System.out.println();

    System.out.println("=====
    ==");
    System.out.println(" PART C: Extrapolation for N = 1,000,000
    (ArrayList case)");

    System.out.println("=====
    ==");
    long baseN = 32000;
    long baseT = timeArrayListDedup((int) baseN);
    double factor = Math.pow(1_000_000.0 / baseN, 2); // O(n^2)
    scaling
    double estimatedMs = baseT * factor;
    System.out.println("Measured ArrayList time at N=32,000      : "
+ baseT + " ms");
    System.out.println("O(n^2) scaling factor to N=1,000,000    : "
+ String.format("%.1f", factor) + "x");
    System.out.println("Estimated ArrayList time at N=1,000,000  : "
+ String.format("%.0f", estimatedMs) + " ms (~"
+ String.format("%.1f", estimatedMs / 60000.0) + "
minutes)");
    }
}

```

4. Output

```
Command Prompt
C:\Users\Joanna\CollectionsBenchmark>javac DuplicateCheckBenchmark.java
C:\Users\Joanna\CollectionsBenchmark>java DuplicateCheckBenchmark
=====
PART A: ArrayList.contains() based de-duplication
(Operation used: contains() -> O(n) scan per call)
=====
N          Time (ms)      Ratio vs prev
4000        10             -
8000        28            2.80x
16000       101           3.61x
32000       427           4.23x
=====
PART B: HashSet.contains() based de-duplication
(Operation used: contains() -> O(1) average per call)
=====
N          Time (ms)      Ratio vs prev
4000         3             -
8000         3            1.00x
16000        8            2.67x
32000       15            1.88x
1000000     343           22.87x
=====
PART C: Extrapolation for N = 1,000,000 (ArrayList case)
=====
Measured ArrayList time at N=32,000      : 417 ms
O(n^2) scaling factor to N=1,000,000     : 976.6x
Estimated ArrayList time at N=1,000,000  : 407227 ms (~6.8 minutes)
C:\Users\Joanna\CollectionsBenchmark>
```

5. Analysis of Results

- Part A (ArrayList.contains()): runtime ratio climbs toward ~4x each time N doubles (2.80x → 3.61x → 4.23x) — this matches $O(n^2)$, because each of the n calls to contains() itself costs $O(n)$.
- Part B (HashSet.contains()): runtime stays close to the growth of N itself (roughly 2x per doubling, and ~23x when N grows ~31x from 32,000 to 1,000,000) — this matches $O(n)$, i.e. $O(1)$ average per call.
- Part C confirms the hypothesis quantitatively: scaling the measured ArrayList time at $N = 32,000$ by the $O(n^2)$ factor ($\approx 977x$) predicts about 6.8 minutes at $N = 1,000,000$, versus the 343 ms actually measured for the HashSet version at the same size.

Root cause: the slowdown is not caused by "one million elements" in general — it is caused by pairing a linear-scan operation (contains() on ArrayList) with a loop that calls it once per element, which multiplies two $O(n)$ factors into $O(n^2)$. The same one million elements processed with an $O(1)$ -average lookup structure (HashSet/HashMap) finishes in well under a second.

6. Recommendation

- 1) Replace ArrayList with HashSet (or HashMap) wherever the operation performed is membership testing (contains) rather than positional access (get by index) or maintaining insertion/sort order.
- 2) Keep ArrayList only where indexed access or order-preserving iteration is actually required, and where lookups by value are rare.

3) For future changes, re-run this same benchmark-and-ratio check on any collection operation placed inside a loop before it reaches production with large data.